Mansurbek Satarov (M17106450)

Professor Rashmi

CS6065 Introduction to Cloud Computing

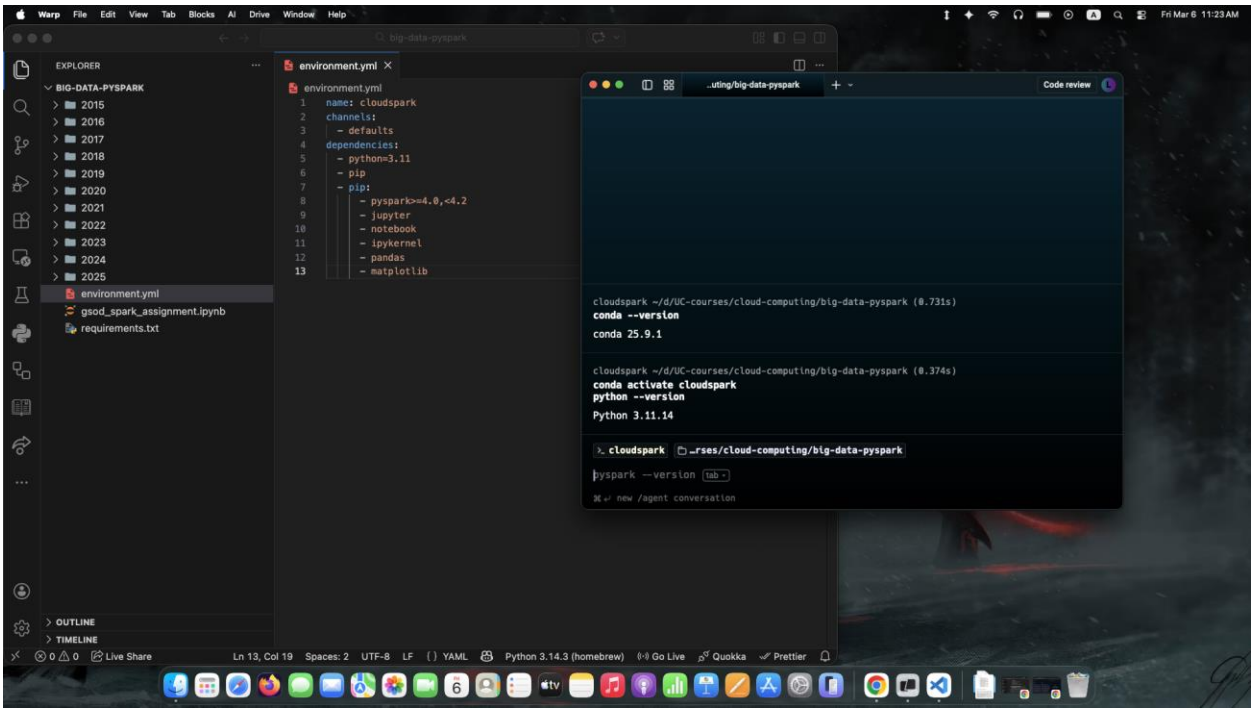
Assignment 4 (Big Data with PySpark using Anaconda & Jupyter notebook)

March 6, 2026

Code on Github: https://github.com/MansurPro/pyspark-weather-analysis

1. Download Anaconda Python, install PySpark, and launch Jupyter Notebook

1.1 Conda setup (I had mini conda in my machine, so I used it directly)



1.2 Download Apache Spark

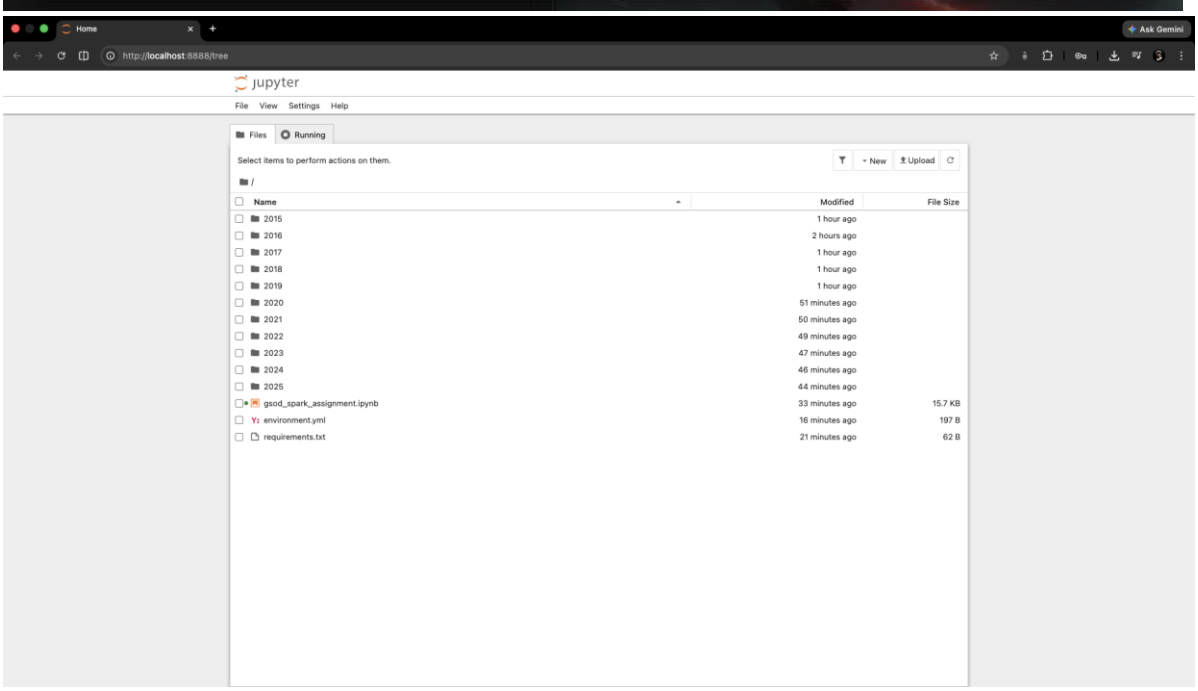
```
cloudspark ~/d/UC-courses/cloud-computing/big-data-pyspark (1.168s)
source ~/.zshrc

base ~/d/UC-courses/cloud-computing/big-data-pyspark (1.854s)
echo $SPARK_HOME
spark-shell --version

/Users/mansur/spark/spark-4.1.1-bin-hadoop3
WARNING: Using incubator modules: jdk.incubator.vector
Welcome to
      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /___/ .__/\_,_/_/ /_/\_\   version 4.1.1
      /_/

Using Scala version 2.13.17, OpenJDK 64-Bit Server VM, 20
Branch HEAD
Compiled by user runner on 2026-01-02T11:55:02Z
Revision c0690c763bafabd08e7079d1137fa0a769a05bae
Url https://github.com/apache/spark
Type --help for more information.

>_ base  …rses/cloud-computing/big-data-pyspark

pyspark --version  tab

⌘↵ new /agent conversation
```

1.3 Install PySpark

```
base ~/d/UC-courses/cloud-computing/big-data-pyspark (0.669s)
conda activate cloudspark
python --version

Python 3.11.14

cloudspark ~/d/UC-courses/cloud-computing/big-data-pyspark (1.348s)
python -c "import pyspark; print(pyspark.__version__)"

4.1.1

>_ cloudspark  …rses/cloud-computing/big-data-pyspark

ls -l environment.yml  tab

⌘↵ new /agent conversation
```

1.4 Launching Jupyter Notebook

```
cloudspark ~/d/UC-courses/cloud-computing/big-data-pyspark (0.057s)
clear

cloudspark ~/d/UC-courses/cloud-computing/big-data-pyspark
jupyter notebook
[I 2026-03-06 11:38:08.067 ServerApp] jupyter_lsp | extension was successfully linked.
[I 2026-03-06 11:38:08.070 ServerApp] jupyter_server_terminals | extension was successfull
y linked.
[I 2026-03-06 11:38:08.071 ServerApp] jupyterlab | extension was successfully linked.
[I 2026-03-06 11:38:08.073 ServerApp] notebook | extension was successfully linked.
[I 2026-03-06 11:38:08.075 ServerApp] Writing Jupyter server cookie secret to /Users/mansu
r/Library/Jupyter/runtime/jupyter_cookie_secret
[I 2026-03-06 11:38:08.308 ServerApp] notebook_shim | extension was successfully linked.
[I 2026-03-06 11:38:08.336 ServerApp] notebook_shim | extension was successfully loaded.
[I 2026-03-06 11:38:08.337 ServerApp] jupyter_lsp | extension was successfully loaded.
[I 2026-03-06 11:38:08.339 ServerApp] jupyter_server_terminals | extension was successfull
y loaded.
[I 2026-03-06 11:38:08.341 LabApp] JupyterLab extension loaded from /Users/mansur/minicond
a3/envs/cloudspark/lib/python3.11/site-packages/jupyterlab
[I 2026-03-06 11:38:08.341 LabApp] JupyterLab application directory is /Users/mansur/minic
onda3/envs/cloudspark/share/jupyter/lab
[I 2026-03-06 11:38:08.342 LabApp] Extension Manager is 'pypi'.
[I 2026-03-06 11:38:08.368 ServerApp] jupyterlab | extension was successfully loaded.
[I 2026-03-06 11:38:08.370 ServerApp] notebook | extension was successfully loaded.
[I 2026-03-06 11:38:08.371 ServerApp] Serving notebooks from local directory: /Users/mansu
r/D/UC-courses/cloud-computing/big-data-pyspark
[I 2026-03-06 11:38:08.371 ServerApp] Jupyter Server 2.17.0 is running at:
[I 2026-03-06 11:38:08.371 ServerApp] http://localhost:8888/tree?token=57a5c66b86a235ad5d9
02de5572299a30d23e4e196845bf3
[I 2026-03-06 11:38:08.371 ServerApp]     http://127.0.0.1:8888/tree?token=57a5c66b86a235a
d5d902de5572299a30d23e4e196845bf3
```



2. Load the CSV files and display the count of each dataset

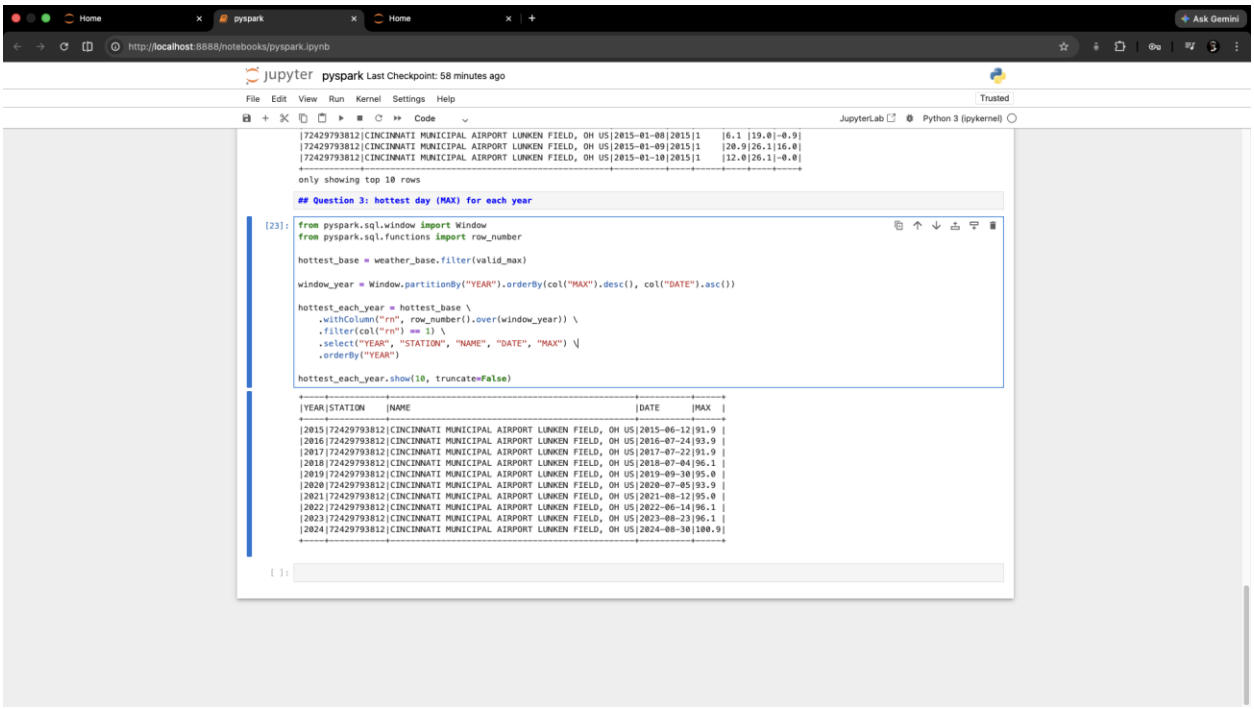
```
[8]: import os

base_path = "."

files = []

for year in range(2015, 2025):

    cincinnati = f"{base_path}/{year}/72429793812.csv"
    florida = f"{base_path}/{year}/99495199999.csv"

    if os.path.exists(cincinnati):
        files.append((year, "Cincinnati", "72429793812", cincinnati))

    if os.path.exists(florida):
        files.append((year, "Florida", "99495199999", florida))

print("Total datasets found:", len(files))

for f in files:
    print(f)
```

```
Total datasets found: 19
```

```
[10]: counts_df = spark.createDataFrame(
    dataset_counts,
    ["YEAR", "CITY", "STATION", "ROW_COUNT"]
)

counts_df.orderBy("YEAR", "CITY").show(25, truncate=False)
```

```
+----+----------+-----------+---------+
|YEAR|CITY      |STATION    |ROW_COUNT|
+----+----------+-----------+---------+
|2015|Cincinnati|72429793812|365      |
|2015|Florida   |99495199999|355      |
|2016|Cincinnati|72429793812|366      |
|2017|Cincinnati|72429793812|365      |
|2017|Florida   |99495199999|283      |
|2018|Cincinnati|72429793812|365      |
|2018|Florida   |99495199999|363      |
|2019|Cincinnati|72429793812|365      |
|2019|Florida   |99495199999|345      |
|2020|Cincinnati|72429793812|366      |
|2020|Florida   |99495199999|365      |
|2021|Cincinnati|72429793812|365      |
|2021|Florida   |99495199999|104      |
|2022|Cincinnati|72429793812|365      |
|2022|Florida   |99495199999|259      |
|2023|Cincinnati|72429793812|365      |
|2023|Florida   |99495199999|276      |
|2024|Cincinnati|72429793812|366      |
|2024|Florida   |99495199999|133      |
+----+----------+-----------+---------+
```

3. Find the hottest day (column MAX) for each year



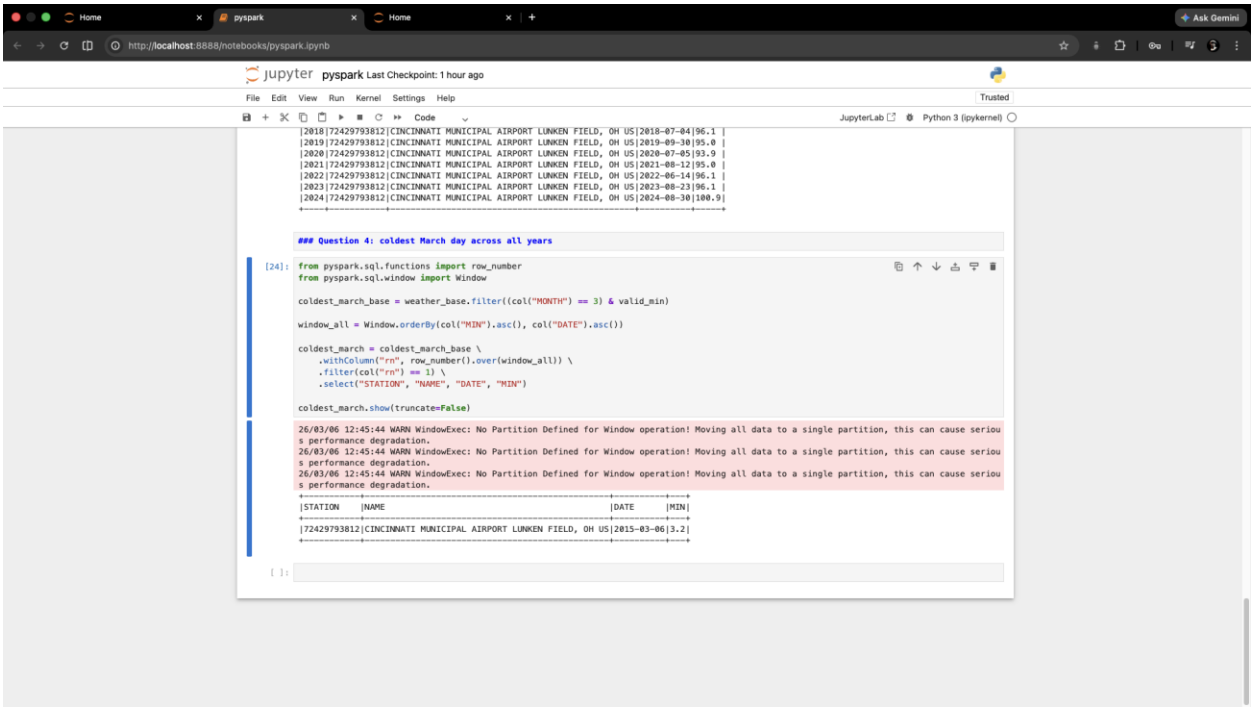
```
|72429793812|CINCINNATI MUNICIPAL AIRPORT LUNKEN FIELD, OH US|2015-01-08|2015|1    |6.1 |19.0|-0.9|
|72429793812|CINCINNATI MUNICIPAL AIRPORT LUNKEN FIELD, OH US|2015-01-09|2015|1    |20.9|26.1|16.0|
|72429793812|CINCINNATI MUNICIPAL AIRPORT LUNKEN FIELD, OH US|2015-01-10|2015|1    |12.0|26.1|-0.0|
+-----------+-----------------------------------------------+----------+----+-----+----+----+----+
only showing top 10 rows
```

```
## Question 3: hottest day (MAX) for each year
```

[23]:
```
from pyspark.sql.window import Window
from pyspark.sql.functions import row_number

hottest_base = weather_base.filter(valid_max)

window_year = Window.partitionBy("YEAR").orderBy(col("MAX").desc(), col("DATE").asc())

hottest_each_year = hottest_base \
    .withColumn("rn", row_number().over(window_year)) \
    .filter(col("rn") == 1) \
    .select("YEAR", "STATION", "NAME", "DATE", "MAX") \
    .orderBy("YEAR")

hottest_each_year.show(10, truncate=False)
```

```
+----+-----------+-----------------------------------------------+----------+----+
|YEAR|STATION    |NAME                                           |DATE      |MAX |
+----+-----------+-----------------------------------------------+----------+----+
|2015|72429793812|CINCINNATI MUNICIPAL AIRPORT LUNKEN FIELD, OH US|2015-06-12|91.9 |
|2016|72429793812|CINCINNATI MUNICIPAL AIRPORT LUNKEN FIELD, OH US|2016-07-24|93.9 |
|2017|72429793812|CINCINNATI MUNICIPAL AIRPORT LUNKEN FIELD, OH US|2017-07-22|91.9 |
|2018|72429793812|CINCINNATI MUNICIPAL AIRPORT LUNKEN FIELD, OH US|2018-07-04|96.1 |
|2019|72429793812|CINCINNATI MUNICIPAL AIRPORT LUNKEN FIELD, OH US|2019-09-30|95.0 |
|2020|72429793812|CINCINNATI MUNICIPAL AIRPORT LUNKEN FIELD, OH US|2020-07-05|93.9 |
|2021|72429793812|CINCINNATI MUNICIPAL AIRPORT LUNKEN FIELD, OH US|2021-08-12|95.0 |
|2022|72429793812|CINCINNATI MUNICIPAL AIRPORT LUNKEN FIELD, OH US|2022-06-14|96.1 |
|2023|72429793812|CINCINNATI MUNICIPAL AIRPORT LUNKEN FIELD, OH US|2023-08-23|96.1 |
|2024|72429793812|CINCINNATI MUNICIPAL AIRPORT LUNKEN FIELD, OH US|2024-08-30|100.9|
+----+-----------+-----------------------------------------------+----------+----+
```

4. Find the coldest day (column MIN) for the month of March across all years (2015-2024)



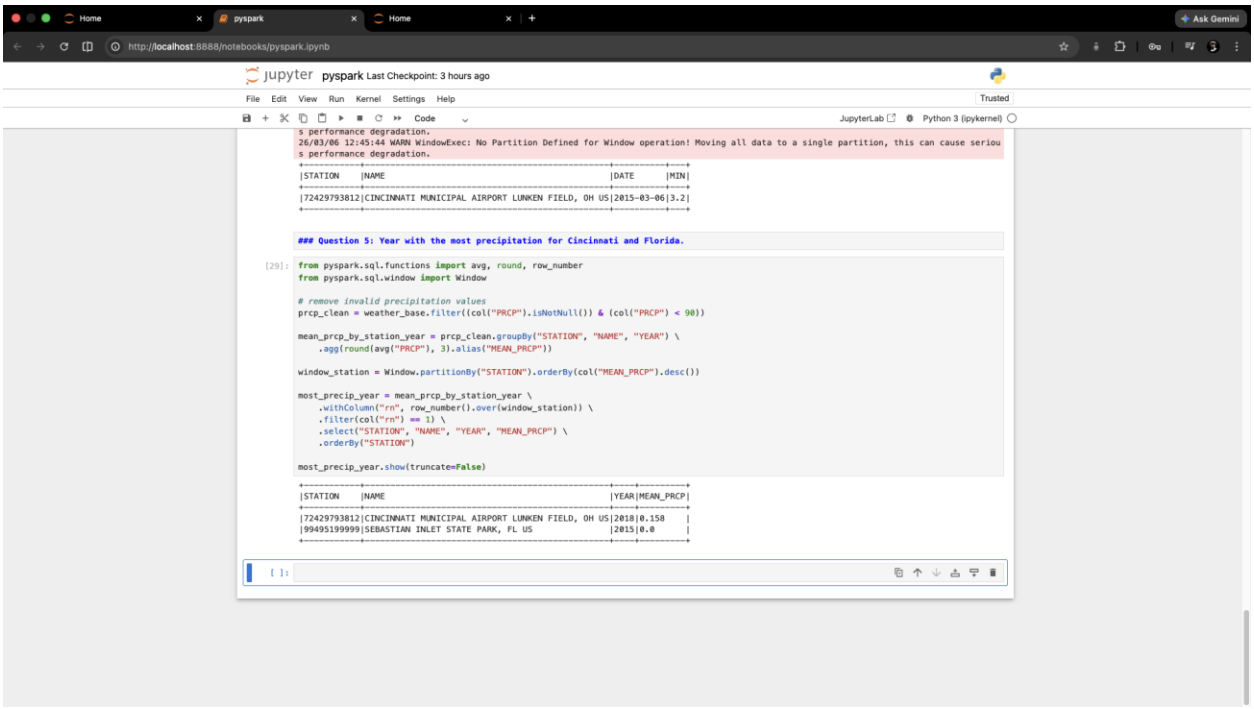
```
|2018|72429793812|CINCINNATI MUNICIPAL AIRPORT LUNKEN FIELD, OH US|2018-07-04|96.1 |
|2019|72429793812|CINCINNATI MUNICIPAL AIRPORT LUNKEN FIELD, OH US|2019-09-30|95.0 |
|2020|72429793812|CINCINNATI MUNICIPAL AIRPORT LUNKEN FIELD, OH US|2020-07-05|93.9 |
|2021|72429793812|CINCINNATI MUNICIPAL AIRPORT LUNKEN FIELD, OH US|2021-08-12|95.0 |
|2022|72429793812|CINCINNATI MUNICIPAL AIRPORT LUNKEN FIELD, OH US|2022-06-14|96.1 |
|2023|72429793812|CINCINNATI MUNICIPAL AIRPORT LUNKEN FIELD, OH US|2023-08-23|96.1 |
|2024|72429793812|CINCINNATI MUNICIPAL AIRPORT LUNKEN FIELD, OH US|2024-08-30|100.9|
+----+-----------+-----------------------------------------------+----------+----+
```

```
### Question 4: coldest March day across all years
```

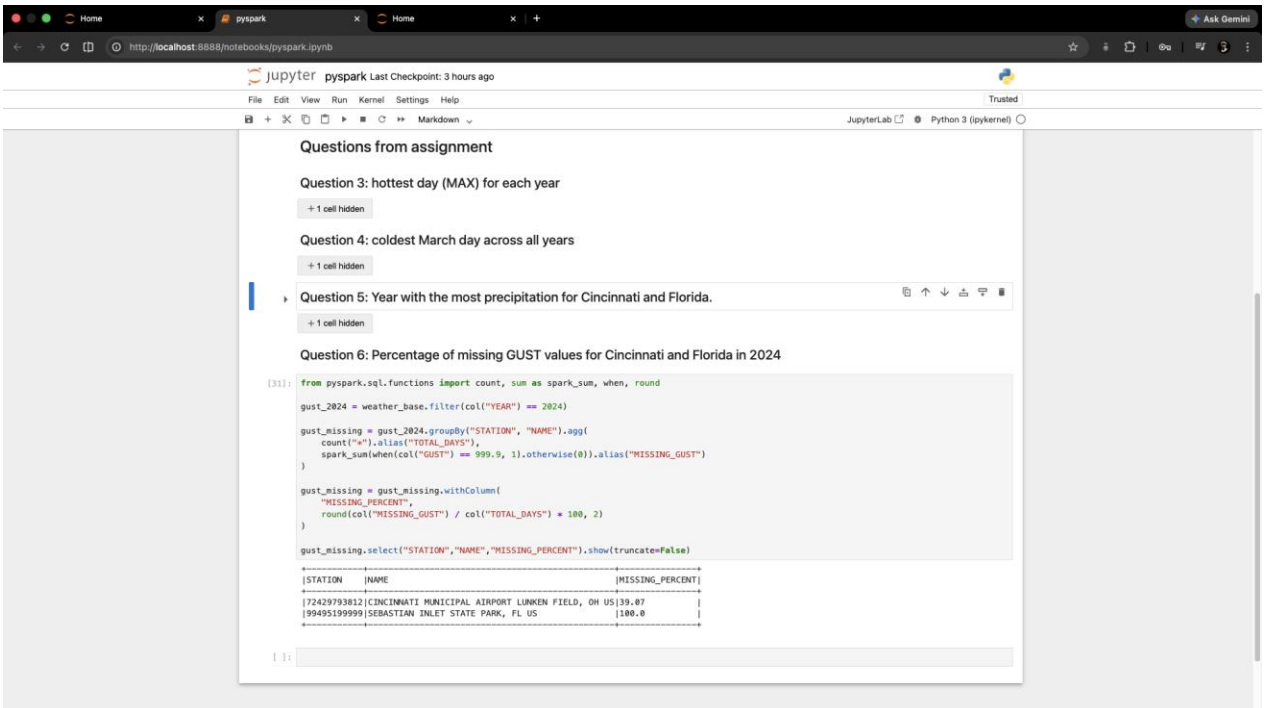
[24]:
```
from pyspark.sql.functions import row_number
from pyspark.sql.window import Window

coldest_march_base = weather_base.filter((col("MONTH") == 3) & valid_min)

window_all = Window.orderBy(col("MIN").asc(), col("DATE").asc())

coldest_march = coldest_march_base \
    .withColumn("rn", row_number().over(window_all)) \
    .filter(col("rn") == 1) \
    .select("STATION", "NAME", "DATE", "MIN")

coldest_march.show(truncate=False)
```

```
26/03/06 12:45:44 WARN WindowExec: No Partition Defined for Window operation! Moving all data to a single partition, this can cause seriou
s performance degradation.
26/03/06 12:45:44 WARN WindowExec: No Partition Defined for Window operation! Moving all data to a single partition, this can cause seriou
s performance degradation.
26/03/06 12:45:44 WARN WindowExec: No Partition Defined for Window operation! Moving all data to a single partition, this can cause seriou
s performance degradation.
+-----------+-----------------------------------------------+----------+---+
|STATION    |NAME                                           |DATE      |MIN|
+-----------+-----------------------------------------------+----------+---+
|72429793812|CINCINNATI MUNICIPAL AIRPORT LUNKEN FIELD, OH US|2015-03-06|3.2|
+-----------+-----------------------------------------------+----------+---+
```

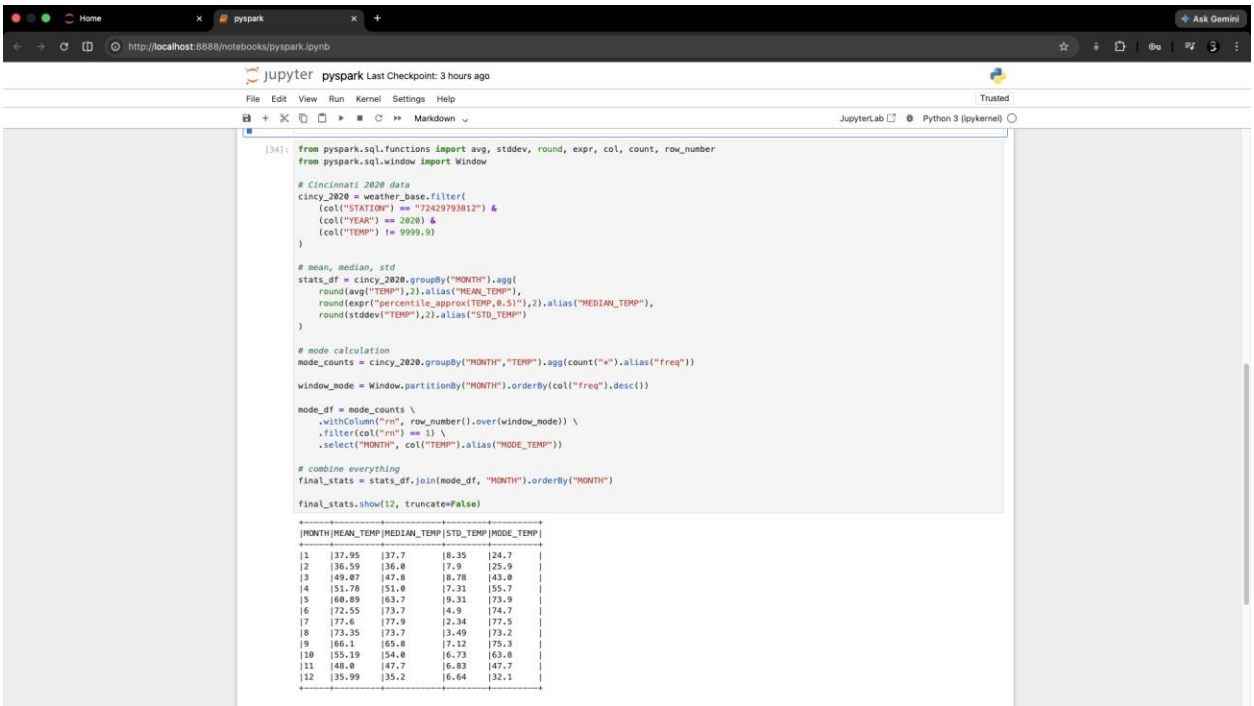
5. Find the year with the most precipitation for Cincinnati and Florida



6. Percentage of missing GUST values for Cincinnati and Florida in 2024

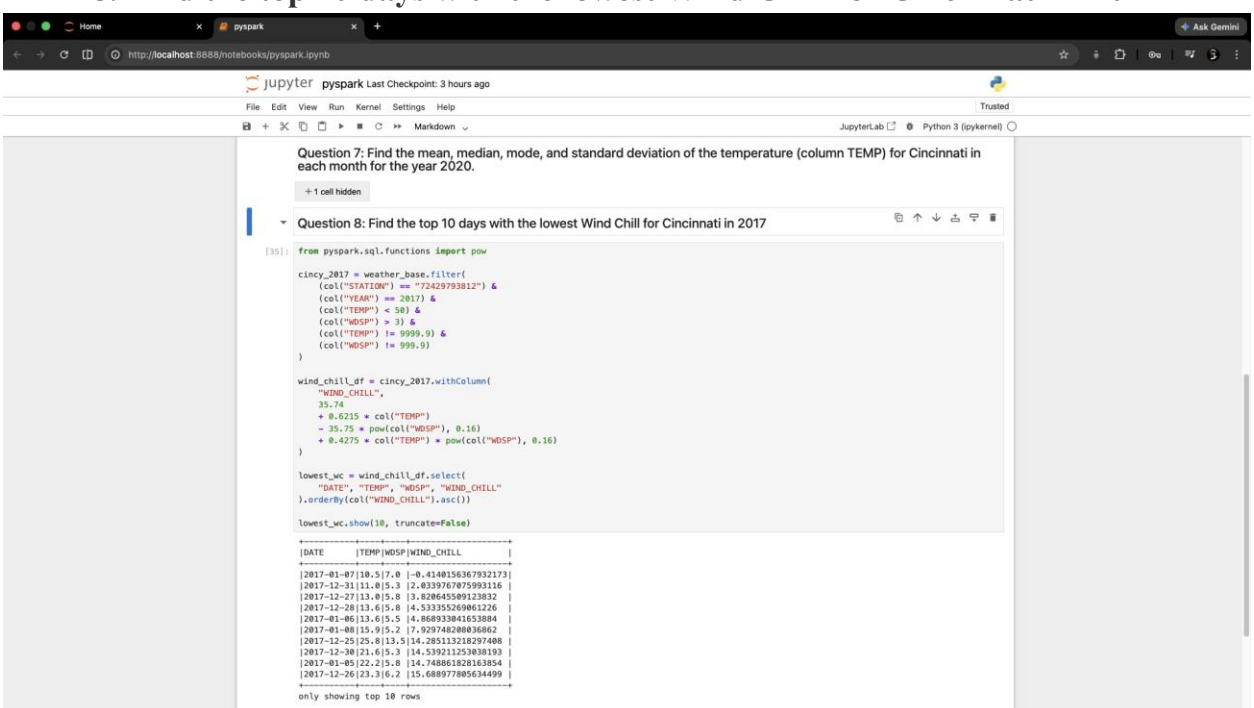


7. Find the mean, median, mode, and standard deviation of the temperature (column TEMP) for Cincinnati in each month for the year 2020



```
[34]: from pyspark.sql.functions import avg, stddev, round, expr, col, count, row_number
      from pyspark.sql.window import Window

      # Cincinnati 2020 data
      cincy_2020 = weather_base.filter(
          (col("STATION") == "72429793812") &
          (col("YEAR") == 2020) &
          (col("TEMP") != 9999.9)
      )

      # mean, median, std
      stats_df = cincy_2020.groupBy("MONTH").agg(
          round(avg("TEMP"),2).alias("MEAN_TEMP"),
          round(expr("percentile_approx(TEMP,0.5)"),2).alias("MEDIAN_TEMP"),
          round(stddev("TEMP"),2).alias("STD_TEMP")
      )

      # mode calculation
      mode_counts = cincy_2020.groupBy("MONTH","TEMP").agg(count("*").alias("freq"))

      window_mode = Window.partitionBy("MONTH").orderBy(col("freq").desc())

      mode_df = mode_counts \
          .withColumn("rn", row_number().over(window_mode)) \
          .filter(col("rn") == 1) \
          .select("MONTH", col("TEMP").alias("MODE_TEMP"))

      # combine everything
      final_stats = stats_df.join(mode_df, "MONTH").orderBy("MONTH")

      final_stats.show(12, truncate=False)

+-----+---------+-----------+--------+---------+
|MONTH|MEAN_TEMP|MEDIAN_TEMP|STD_TEMP|MODE_TEMP|
+-----+---------+-----------+--------+---------+
|1    |37.95    |37.7       |8.35    |24.7     |
|2    |36.59    |36.0       |7.9     |25.9     |
|3    |49.07    |47.8       |8.78    |43.0     |
|4    |51.78    |51.0       |7.31    |55.7     |
|5    |60.89    |63.7       |9.31    |73.9     |
|6    |72.55    |73.7       |4.9     |74.7     |
|7    |77.6     |77.9       |2.34    |77.5     |
|8    |73.35    |73.7       |3.49    |73.2     |
|9    |66.1     |65.8       |7.12    |75.3     |
|10   |55.19    |54.0       |6.73    |63.8     |
|11   |48.0     |47.7       |6.83    |47.7     |
|12   |35.99    |35.2       |6.64    |32.1     |
+-----+---------+-----------+--------+---------+
```

8. Find the top 10 days with the lowest Wind Chill for Cincinnati in 2017



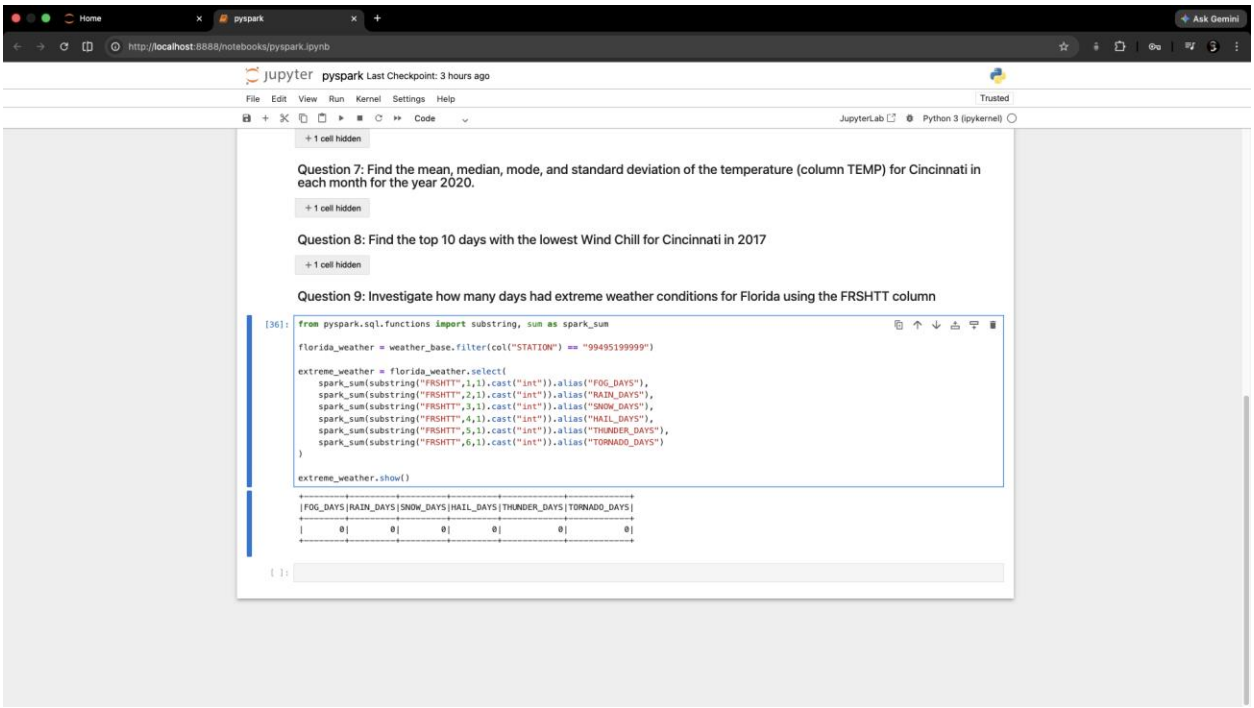
Question 7: Find the mean, median, mode, and standard deviation of the temperature (column TEMP) for Cincinnati in each month for the year 2020.

+ 1 cell hidden

Question 8: Find the top 10 days with the lowest Wind Chill for Cincinnati in 2017

```
[35]: from pyspark.sql.functions import pow

      cincy_2017 = weather_base.filter(
          (col("STATION") == "72429793812") &
          (col("YEAR") == 2017) &
          (col("TEMP") < 50) &
          (col("WDSP") > 3) &
          (col("TEMP") != 9999.9) &
          (col("WDSP") != 999.9)
      )

      wind_chill_df = cincy_2017.withColumn(
          "WIND_CHILL",
          35.74
          + 0.6215 * col("TEMP")
          - 35.75 * pow(col("WDSP"), 0.16)
          + 0.4275 * col("TEMP") * pow(col("WDSP"), 0.16)
      )

      lowest_wc = wind_chill_df.select(
          "DATE", "TEMP", "WDSP", "WIND_CHILL"
      ).orderBy(col("WIND_CHILL").asc())

      lowest_wc.show(10, truncate=False)

+----------+----+----+------------------+
|DATE      |TEMP|WDSP|WIND_CHILL        |
+----------+----+----+------------------+
|2017-01-07|10.5|7.0 |-0.4140156367932173|
|2017-12-31|11.0|5.3 |2.0339767075993116 |
|2017-12-27|13.0|5.8 |3.820645509123832  |
|2017-12-28|13.6|5.8 |4.533355269061226  |
|2017-01-06|13.6|5.5 |4.868933041653884  |
|2017-01-08|15.9|5.2 |7.929748208036862  |
|2017-12-25|25.8|13.5|14.285113218297408 |
|2017-12-30|21.6|5.3 |14.539211253038193 |
|2017-01-05|22.2|5.8 |14.748861828163854 |
|2017-12-26|23.3|6.2 |15.688977805634499 |
+----------+----+----+------------------+
only showing top 10 rows
```

9. Investigate how many days had extreme weather conditions for Florida (fog, rain, snow, etc.) using the FRSHTT column



Question 7: Find the mean, median, mode, and standard deviation of the temperature (column TEMP) for Cincinnati in each month for the year 2020.

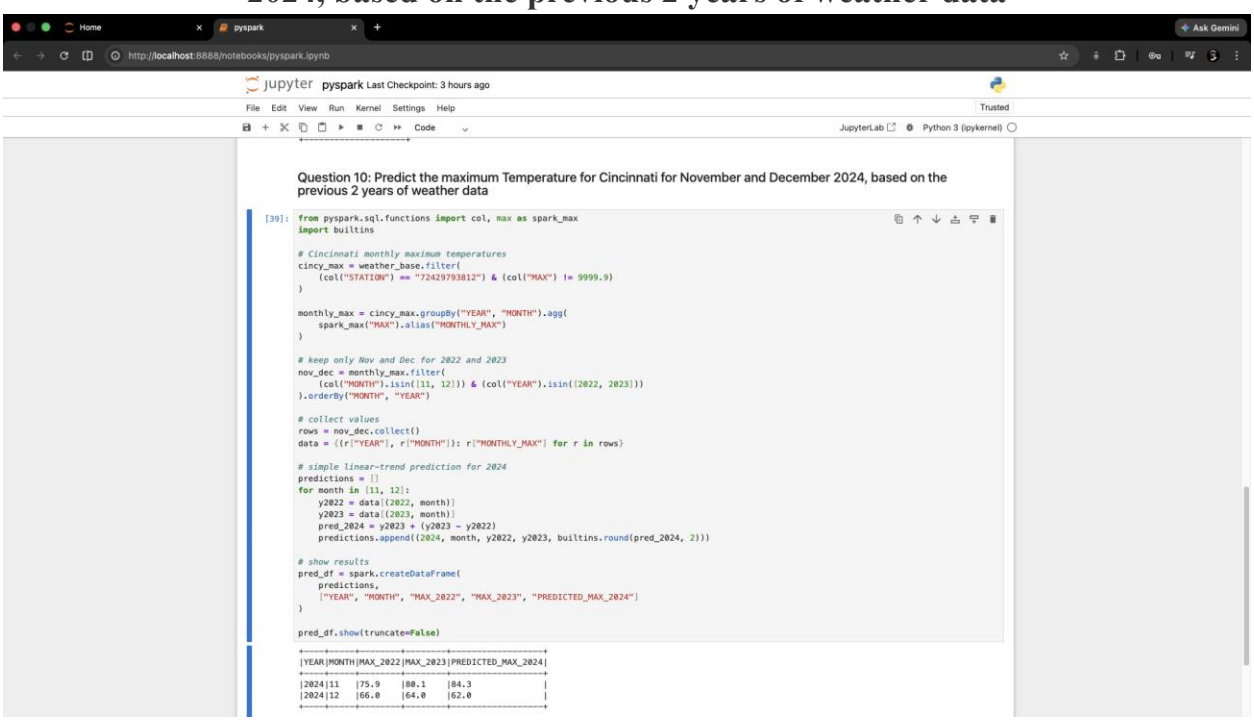
Question 8: Find the top 10 days with the lowest Wind Chill for Cincinnati in 2017

Question 9: Investigate how many days had extreme weather conditions for Florida using the FRSHTT column

```
[36]: from pyspark.sql.functions import substring, sum as spark_sum

florida_weather = weather_base.filter(col("STATION") == "99495199999")

extreme_weather = florida_weather.select(
    spark_sum(substring("FRSHTT",1,1).cast("int")).alias("FOG_DAYS"),
    spark_sum(substring("FRSHTT",2,1).cast("int")).alias("RAIN_DAYS"),
    spark_sum(substring("FRSHTT",3,1).cast("int")).alias("SNOW_DAYS"),
    spark_sum(substring("FRSHTT",4,1).cast("int")).alias("HAIL_DAYS"),
    spark_sum(substring("FRSHTT",5,1).cast("int")).alias("THUNDER_DAYS"),
    spark_sum(substring("FRSHTT",6,1).cast("int")).alias("TORNADO_DAYS")
)

extreme_weather.show()
```

```
+--------+---------+---------+---------+-------------+-------------+
|FOG_DAYS|RAIN_DAYS|SNOW_DAYS|HAIL_DAYS|THUNDER_DAYS|TORNADO_DAYS|
+--------+---------+---------+---------+-------------+-------------+
|       0|        0|        0|        0|            0|            0|
+--------+---------+---------+---------+-------------+-------------+
```

10. Predict the maximum Temperature for Cincinnati for November and December 2024, based on the previous 2 years of weather data



Question 10: Predict the maximum Temperature for Cincinnati for November and December 2024, based on the previous 2 years of weather data

```
[39]: from pyspark.sql.functions import col, max as spark_max
import builtins

# Cincinnati monthly maximum temperatures
cincy_max = weather_base.filter(
    (col("STATION") == "72429793812") & (col("MAX") != 9999.9)
)

monthly_max = cincy_max.groupBy("YEAR", "MONTH").agg(
    spark_max("MAX").alias("MONTHLY_MAX")
)

# keep only Nov and Dec for 2022 and 2023
nov_dec = monthly_max.filter(
    (col("MONTH").isin([11, 12])) & (col("YEAR").isin([2022, 2023]))
).orderBy("MONTH", "YEAR")

# collect values
rows = nov_dec.collect()
data = {(r["YEAR"], r["MONTH"]): r["MONTHLY_MAX"] for r in rows}

# simple linear-trend prediction for 2024
predictions = []
for month in [11, 12]:
    y2022 = data[(2022, month)]
    y2023 = data[(2023, month)]
    pred_2024 = y2023 + (y2023 - y2022)
    predictions.append((2024, month, y2022, y2023, builtins.round(pred_2024, 2)))

# show results
pred_df = spark.createDataFrame(
    predictions,
    ["YEAR", "MONTH", "MAX_2022", "MAX_2023", "PREDICTED_MAX_2024"]
)

pred_df.show(truncate=False)
```

```
+----+-----+--------+--------+-------------------+
|YEAR|MONTH|MAX_2022|MAX_2023|PREDICTED_MAX_2024|
+----+-----+--------+--------+-------------------+
|2024|11   |75.9    |80.1    |84.3               |
|2024|12   |66.0    |64.0    |62.0               |
+----+-----+--------+--------+-------------------+
```